

Perfect. Now we are entering the **System Design** part. This is the most important section because once this is finalized, coding becomes much easier.

Engineering Design Document (EDD-01)

Environment Support

Section 13 - Database Design

13.1 Current Database

Current simplified structure

```
Organization
  |
Project
  |
FeatureFlag
```

FeatureFlag currently stores

```
Feature Name
Description
Enabled
Rollout Percentage
Kill Switch
...
```

Problem

This mixes two different concepts.

Identity

```
Feature Name

Description

Owner

Tags
```

and

Configuration

Enabled

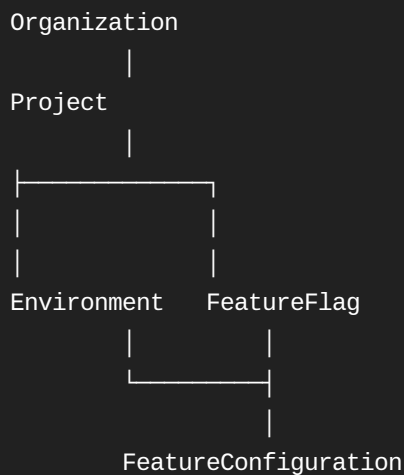
Rollout

Variables

Rules

These should not be stored together.

13.2 Proposed Database



Notice

Environment and FeatureFlag are independent.

FeatureConfiguration joins them together.

This is the heart of the design.

13.3 Why FeatureConfiguration?

Let's consider one feature.

Dark Mode

Suppose we have

Development

Staging

Production

Should we create

Dark Mode Dev

Dark Mode Stage

Dark Mode Prod

No.

Instead

Feature

Dark Mode

has

Development Configuration

Staging Configuration

Production Configuration

This avoids duplication.

13.4 Environment Collection

TypeScript

Environment

Field	Type	Required	Description
_id	ObjectId	Yes	Primary Key
projectId	ObjectId	Yes	Parent Project
name	String	Yes	Development

Field	Type	Required	Description
slug	String	Yes	development
description	String	No	Optional
color	String	Yes	UI Badge
order	Number	Yes	Display Order
isDefault	Boolean	Yes	Default Environment
createdBy	ObjectId	Yes	User
createdAt	Date	Yes	Timestamp
updatedAt	Date	Yes	Timestamp

Why every field?

projectId

One project



Many environments

slug

Instead of

Development

SDKs should use

development

Slug is stable.

Name can change.

order

Without order

MongoDB returns

```
Production
Development
Staging
```

Random.

Order keeps UI consistent.

color

Pure UI.

Example

```
Development
Green
Production
Red
```

isDefault

Instead of checking

```
name == Production
```

we simply check

```
isDefault == true
```

Much cleaner.

13.5 FeatureFlag Collection

FeatureFlag becomes smaller.

TypeScript

FeatureFlag

Field	Description
name	Feature Name
key	SDK Key Name
description	Description
owner	User
tags	Labels
projectId	Parent Project
archived	Boolean
createdBy	User
createdAt	Timestamp

Notice

No rollout.

No enabled.

No rules.

No variables.

This is intentional.

FeatureFlag becomes only metadata.

13.6 FeatureConfiguration Collection

This is the most important collection.

Field	Description
featureId	Parent Feature
environmentId	Environment
enabled	Enabled
rolloutPercentage	0-100
killSwitch	Emergency Disable
targetingRules	Rules
variables	JSON Variables
updatedBy	Last Editor
createdAt	Timestamp
updatedAt	Timestamp

Why FeatureConfiguration?

One Feature



Many Configurations

Example

Dark Mode

Development

Enabled

100%

Staging

Enabled

50%

Production

Enabled

5%

Everything remains independent.

13.7 Relationship

Project

|

|----- FeatureFlag

|

|

|

|

|----- Environment

|

|

FeatureConfiguration

Relationship

One Project

↓

Many Features

↓

Many Environments

↓

FeatureConfiguration

belongs to

One Feature

AND

One Environment

13.8 Unique Constraints

Environment

(projectId, slug)

Cannot have

Development

Development

inside one project.

FeatureFlag

(projectId, key)

Cannot create

dark-mode

dark-mode

FeatureConfiguration

(featureId, environmentId)

Exactly one configuration.

Never duplicates.

13.9 Indexes

Environment

projectId

FeatureFlag

projectId

key

FeatureConfiguration

featureId

environmentId

(featureId, environmentId)

This keeps SDK evaluation fast.

13.10 Cascade Rules

Suppose

Project deleted

↓

Delete

Environment

↓

Delete

Feature

↓

Delete

FeatureConfiguration

No orphan documents should remain.

13.11 Creation Flow

When Project created

Create Project

↓

Development

↓

Staging

↓

Production

When Feature created

Create Feature

↓

Fetch Environments

↓

For each Environment

↓

Create Configuration

Immediately.

No lazy creation.

13.12 Future Compatibility

Because of this design

later we can add

Environment

SDK Keys

Analytics

Webhook

Deployment History

Secrets

Cache

Rate Limits

without touching FeatureFlag.

That is why Environment deserves its own collection.

13.13 Things Developer Must NOT Do

- ✗ Do not duplicate Feature documents.
- ✗ Do not store rollout inside FeatureFlag.
- ✗ Do not hardcode environment names in business logic.
- ✗ Never use environment name for comparisons.

Use

environmentId

or

slug

instead.

Deliverables for Backend Developer

Create

- Environment Model
- Environment Repository
- Environment Service
- Environment Controller
- Environment Routes

Modify

- Project Creation Service (auto-create default environments)
- Feature Creation Service (auto-create configurations)

Create

- FeatureConfiguration Model
- Repository
- CRUD Service

Validation

- Unique slug
 - Cannot delete default environments
 - Validate project ownership
 - Validate environment belongs to project
-

My suggestion

This document is now becoming a proper **Technical Design Document (TDD)** rather than just notes. I think we should keep that standard for every remaining module.

One more improvement I'd like to make: after we finish each module, we should also produce **sequence diagrams**, **ER diagrams**, and **API contracts**. Those artifacts are what software engineering courses usually expect, and they'll also make it much easier for your teammates to implement the feature consistently.

I also have one recommendation before we move further: while reviewing your repository, I noticed you already have a good architecture (controller → service → repository). I think every new module should **strictly follow the same layering**. We should explicitly document which layer is responsible for what, so teammates don't accidentally put business logic in controllers or database

queries in services. That consistency will keep the codebase maintainable as the team grows.